

Exercise No. 1. Parent-Child process creation

ROHITH KANNA GV
12803214 A- 44

AIM:

To create a child process using fork system call and use pipe for interaction between parent and child

Procedure / Algorithm

- Develop the parent process with code for calls to fork and pipe
- Child process created as a result of fork ()
- Write a message into pipe under parent part of the code
- Suspend parent process to invoke child
- Reads the message from the pipe under the child part of the code
- Parent and child terminates

1a. Write a program to create of a child process using fork, trace the process ids and study about orphan, zombie and daemon process

parentCHILD.C

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    pid = fork();
    if(pid == -1)
    {printf("Fork Failed\n");}
    else if(pid == 0)
    {
        printf("\n----- Child Process ----- \n");
        printf("Child PID : %d\n", getpid());
        printf("Child's Parent PID : %d\n", getppid());
    }else
    {
        printf("\n----- Parent Process ----- \n");
        printf("Parent PID : %d\n", getpid());
        printf("Parent's Parent PID : %d\n", getppid());
        wait(NULL);
    }
}
```

```

    }

    return 0;
}

guest-h4u9fg@cnlab:~$ gedit parentChild.c
guest-h4u9fg@cnlab:~$ gcc parentChild.c
guest-h4u9fg@cnlab:~$ ./a.out

```

```

----- Parent Process -----
Parent PID : 6876
Parent's Parent PID : 2707

----- Child Process -----
Child PID : 6877
Child's Parent PID : 6876

```

orphan.C

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid;
    pid = fork();

    if(pid > 0)
    {
        printf("Parent Process\n");
        printf("Parent PID : %d\n", getpid());
        exit(0);          // Parent exits immediately
    }else if(pid == 0)
    {
        sleep(5);          // Child waits
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("New Parent PID : %d\n", getppid());
    }
    return 0;
}

```

```

guest-h4u9fg@cnlab:~$ gedit orphan.c
guest-h4u9fg@cnlab:~$ gcc orphan.c
guest-h4u9fg@cnlab:~$ ./a.out
Parent Process
Parent PID : 6990
guest-h4u9fg@cnlab:~$
Child Process
Child PID : 6991
New Parent PID : 1305

```

zombie.C

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid;
    pid = fork();
    if(pid > 0)
    {
        printf("Parent Process\n");
        printf("Parent PID : %d\n", getpid());
        sleep(10);    // Parent waits
    }
    else if(pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID : %d\n", getpid());
        exit(0);      // Child exits immediately
    }
    return 0;
}

```

```

guest-h4u9fg@cnlab:~$ gedit zombie.c
guest-h4u9fg@cnlab:~$ gcc zombie.c
guest-h4u9fg@cnlab:~$ ./a.out
Parent Process
Parent PID : 7083
Child Process
Child PID : 7084

```

parentChildComm.C(UNIDIRECTIONAL PARENT->CHILD)

```

include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>

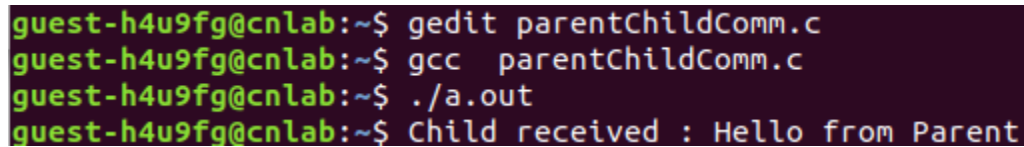
```

```

int main()
{
    int pipe_fd[2];
    char msg[100];
    pipe(pipe_fd);
    pid_t pid = fork();
    if(pid == 0)
    {
        close(pipe_fd[1]);          // Close write end
        read(pipe_fd[0], msg, sizeof(msg));
        printf("Child received : %s\n", msg);
        close(pipe_fd[0]);
    }
    else
    {
        close(pipe_fd[0]);          // Close read end
        char text[] = "Hello from Parent";
        write(pipe_fd[1], text, sizeof(text));
        close(pipe_fd[1]);
    }

    return 0;}

```



```

guest-h4u9fg@cnlab:~$ gedit parentChildComm.c
guest-h4u9fg@cnlab:~$ gcc parentChildComm.c
guest-h4u9fg@cnlab:~$ ./a.out
guest-h4u9fg@cnlab:~$ Child received : Hello from Parent

```

parentChildBiComm.C(BIDIRECTIONAL PARENT<->CHILD)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main()
{
    int fd1[2];
    int fd2[2];
    char fixed_str[] = "Happy ";
    char input_str[100];

    if (pipe(fd1) == -1)
    {
        fprintf(stderr, "Pipe Failed");
        return 1;
    }

```

```

if (pipe(fd2) == -1)
{
    fprintf(stderr, "Pipe Failed");
    return 1;
}

printf("Enter a word: ");
scanf("%s", input_str);

pid_t p = fork();

if (p < 0)
{
    fprintf(stderr, "Fork Failed");
    return 1;
}
else if (p > 0)
{
    char concat_str[100];
    close(fd1[0]);
    write(fd1[1], input_str, strlen(input_str) + 1);
    close(fd1[1]);
    wait(NULL);
    close(fd2[1]);
    read(fd2[0], concat_str, sizeof(concat_str));
    printf("Concatenated String: %s\n", concat_str);
    close(fd2[0]);
}
else
{
    char concat_str[100];
    close(fd1[1]);
    read(fd1[0], concat_str, sizeof(concat_str));

    int k = strlen(concat_str);

    for (int i = 0; i < strlen(fixed_str); i++)
        concat_str[k++] = fixed_str[i];

    concat_str[k] = '\0';

    close(fd1[0]);
    close(fd2[0]);
    write(fd2[1], concat_str, strlen(concat_str) + 1);
    close(fd2[1]);
}

```

```
    return 0;
}
```

```
guest-h4u9fg@cnlab:~$ gedit parentChildBiComm.c
guest-h4u9fg@cnlab:~$ gcc parentChildBiComm.c
guest-h4u9fg@cnlab:~$ ./a.out
Enter a word : Rohith
Concatenated String : RohithHappy
```

1b. Write a program to create of a child process using fork, and establish communication between parent, child and multiple children using pipes

siblings.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    char s1[100], s2[100];
    pid_t id, pid;
    int id1[2], id2[2];

    if (pipe(id1) == -1)
    {
        printf("Pipe 1 failed...\n");
        return 0;
    }
    if (pipe(id2) == -1)
    {
        printf("Pipe 2 failed...\n");
        return 0;
    }

    id = fork();

    if (id == -1)
    {
        printf("Child 1 not created...\n");
        return 0;
    }
    else if (id == 0)
```

```

{
    close(id1[0]);
    close(id2[1]);
    char msg1[] = "This is Child 1...";
    write(id1[1], msg1, sizeof(msg1));
    read(id2[0], s1, sizeof(s1));
    printf("Child 1 reads: \"%s\" from Child 2\n", s1);
    close(id1[1]);
    close(id2[0]);
    exit(0);
}
else
{
    pid = fork();

    if (pid == -1)
    {
        printf("Child 2 not created...\n");
        return 0;
    }
    else if (pid == 0)
    {
        close(id1[1]);
        close(id2[0]);
        char msg2[] = "This is Child 2...";
        read(id1[0], s2, sizeof(s2));
        printf("Child 2 reads: \"%s\" from Child 1\n", s2);
        write(id2[1], msg2, sizeof(msg2));
        close(id1[0]);
        close(id2[1]);
        exit(0);
    }
    else
    {
        close(id1[0]);
        close(id1[1]);
        close(id2[0]);
        close(id2[1]);
        wait(NULL);
        wait(NULL);
        printf("Final Parent Process...\n");
    }
}

return 0;
}

```

```
guest-h4u9fg@cnlab:~$ gedit siblings.c
guest-h4u9fg@cnlab:~$ gcc siblings.c
guest-h4u9fg@cnlab:~$ ./a.out
Child 2 reads : "This is Child 1..." from Child 1
Child 1 reads : "This is Child 2..." from Child 2
Final Parent Process...
```

RESULTS

Thus, the program to create a parent and child process using the `fork()` system call was implemented successfully. The process IDs were obtained using `getpid()` and `getppid()`, and the creation of parent and child processes was verified.